ChatV2 Developer Guide

1. Fundamentals

The Agent Loop

An agent is an LLM running in a loop. It reads messages, decides what to do, and repeats until it has an answer.

```
                    +-------------------+
                    |                   |
                    v                   |
  User  --->  [ LLM Agent ]  -- calls tool -->  [ Tool ]  -- result back --+
  Message     |            |
              |  "I have   |
              |  enough    |
              |  info now" |
              |            |
              +-----+------+
                    |
                    v
              AI Response
              (streamed to user)
```

This is called a **ReAct loop** (Reason + Act). The LLM can call zero, one, or many tools before responding. It decides based on what it knows.

Messages

Every conversation is a list of messages. There are 4 types:

```
SystemMessage     "You are the Campaign Assistant. You have these tools..."
                  (Set once. Defines the agent's personality and rules.)

HumanMessage      "How are my campaigns performing?"
                  (The user's input.)

AIMessage         "Let me check your campaign data."
                  (The LLM's response. May include tool_calls.)

ToolMessage       '{"summary": "3 campaigns found", "data": [...]}'
                  (Result of a tool call. Goes back to the LLM.)
```

The LLM sees ALL of these in order. The system prompt shapes behavior. Tool messages feed data back into the loop.

What Is a Tool?

A tool is a function the LLM can call. It has two parts the LLM cares about:

```
1. The DESCRIPTION (docstring)  -->  LLM reads this to decide WHEN to call it
2. The PARAMETERS (function args) --> LLM fills these in based on conversation
```

And two parts that come back:

```
3. summary + data               -->  Goes back to the LLM (for reasoning)
4. ui_blocks                    -->  Goes to the frontend (for the human)
```

The LLM never sees ui_blocks. The human sees both the LLM's text AND the blocks.

Example: anatomy of a tool

```python
@tool("get_campaign_details")                          # <-- name the LLM calls
async def _get_campaign_details(campaign_id: str       # <-- LLM fills this in
) -> dict:
    """Get performance details for a specific campaign. # <-- LLM reads this to
    Use when the user asks about a particular campaign   #     decide when to call
    by name or ID."""

    data = await fetch_campaign(campaign_id)            # <-- your logic

    return ToolResponse(
        summary=f"Campaign '{data['name']}': "          # <-- LLM gets this text
                f"{data['clicks']} clicks, "
                f"${data['spend']} spent",
        data=data,                                      # <-- LLM gets this as JSON
        ui_blocks=[detail_block.create(data=...)],      # <-- human sees this card
    ).model_dump()
```

What Is an MCP Tool?

Same thing, but the function lives on an external server (like a BigQuery sidecar). The framework wraps it as an `AgenticTool` automatically. From the LLM's perspective, MCP tools and native tools are identical.

What Is a Workflow?

A workflow is a **deterministic pipeline** of steps. Unlike tools (where the LLM decides each action), workflows run a fixed sequence:

```
Tools (LLM-driven):           Workflow (SDK-driven):

  LLM -> tool A                 LLM -> trigger tool
  LLM -> tool B (LLM chose)         -> step 1 (automatic)
  LLM -> tool A again                -> step 2 (automatic)
  LLM -> responds                    -> step 3 (HITL pause)
                                      -> step 4 (automatic)
                                      -> done, back to LLM
```

Use workflows for multi-step operations: "create a campaign" (research -> analyze -> plan -> build -> save).

The frontend shows a live progress card with step statuses:

```
  Creating media plan for nike.com
  [done]    Research website         12s
```

```
  [done]    Analyze market           8s
  [active]  Generate plan            ...
  [pending] Save campaigns
```

What Is a Block?

A block sends rich UI from the backend to the frontend. Instead of the LLM saying "here's a table of data," you send structured data that renders as a proper component.

```
Backend emits:                      Frontend renders:
{                                   +---------------------------+
  type: "campaign_overview",  --->  | Campaign Overview         |
  data: {                           | +-------+--------+------+ |
    campaigns: [...],               | | Name  | Clicks | ROI  | |
    total_spend: 50000              | | Nike  | 12.4k  | 145% | |
  },                                | +-------+--------+------+ |
  display: "inline"                 +---------------------------+
}
```

The `type` string is the only coupling. Backend defines a `BlockSpec`, frontend registers a component in `BLOCK_REGISTRY` with the same key.

Display modes control where the block appears:

- **inline** -- directly in the chat message
- **sidebar** -- clickable card -> opens right panel
- **modal** -- clickable card -> opens dialog
- **fullscreen** -- clickable card -> takes full viewport

What Is HITL?

Human-In-The-Loop pauses execution and asks the user a question. Three types:

```
confirmation    "Approve budget of $50k?"         [Approve] [Cancel]

form            "Review settings"                  Budget: [____]
                                                   Start:  [____]
                                                   [Submit] [Cancel]

choice          "Pick a strategy"                  ( ) Aggressive
                                                   ( ) Balanced
                                                   ( ) Conservative
```

The graph pauses (via LangGraph `interrupt()`), the frontend renders the UI, the user responds, and execution resumes.

What Is the System Prompt?

The system prompt is the most important text in your agent. It tells the LLM:

1. **Who it is** -- "You are the Campaign Assistant"
2. **What tools it has** -- and crucially, WHEN to use each one
3. **Rules** -- response format, things to avoid, tone

```
SYSTEM_PROMPT = """You are the Campaign Assistant for the Accelerate platform.

## Tools
- **create_media_plan**: Use ONLY when the user wants to CREATE a new campaign.
  Always ask for the website URL and budget first.
- **get_campaign_details**: Use when asking about a SPECIFIC existing campaign.
- **generate_image**: Use when the user wants to create ad visuals.

## Rules
- Never fabricate data. If a tool returns empty, say so.
- Keep answers concise unless the user asks for detail.
- Always confirm before triggering create_media_plan (it's a long operation).
"""
```

The system prompt + tool docstrings work together. The system prompt says "when to use tool X." The tool docstring says "what tool X does." Both matter.

The Full Picture

```
AgentConfig = {
    system_prompt     what the LLM knows and how it behaves
    tools             functions the LLM can call (single-shot)
    workflows         multi-step pipelines the LLM can trigger
    model             which LLM to use ("sonnet", "gpt", etc.)
    dynamic_context   per-request additions to the prompt
}
```

At startup, each `AgentConfig` compiles into a LangGraph:

```
  START
    |
    v
  [Agent Node]  <-- LLM with system prompt + tool descriptions
    |
    |-- tool call for "get_details" --> [StreamingToolNode] --> back to Agent
    |-- tool call for "create_plan" --> [Workflow Sub-Graph] --> back to Agent
    |-- no tool call ----------------> END (stream AI response)
```

Dual Storage

The platform has two separate storage systems for two different purposes:

```
1. LangGraph Checkpointer (Postgres)     2. Chat Persistence (db-service REST)
   "Where was I?"                            "What happened?"

   Stores graph execution state              Stores messages + blocks for the UI
   Enables multi-turn memory                 Enables page reload, history, sharing
   Per-agent Postgres database               Shared accelerate-db-service
   Managed by LangGraph internally           Managed by api/chat/persistence.py
```

Checkpointer -- LangGraph saves its state (messages, tool results, workflow progress) to Postgres after each node. This is how the agent "remembers" prior turns and how HITL resume works. You never touch this directly.

Chat Persistence -- After each stream completes, `api/chat/orchestration.py` persists the assistant's message (text + blocks + workflow state) to `accelerate-db-service`. This is what the frontend loads on page refresh via `GET /messages`.

You don't need to worry about either storage when building tools or workflows. The framework handles both automatically.

The Streaming Pipeline

When the user sends a message, here's how it flows through the system:

```
User message
  |
  v
[api/chat/routes.py]         Persists user message to db-service
  |
  v
[api/chat/orchestration.py]  Runs the graph, projects events, manages SSE
  |
  |  For each LangGraph chunk:
  |    1. project_chunk()     Converts raw LangGraph output -> AG-UI events
  |    2. StreamReducer       Accumulates text + blocks + workflow state
  |    3. sse_encode()        Serializes to SSE wire format: "data: {...}\n\n"
  |
  v
[SSE stream to browser]      Frontend Redux processes each event in real-time
  |
  v
[_flush_snapshot()]           After stream ends: persists assistant message
                              + workflow state to db-service
```

StreamEmitter (`core/stream_emitter.py`) -- Used INSIDE tools and workflows to emit events during execution. When your tool calls `emitter.block(...)` or the workflow SDK calls `emitter.activity_delta(...)`, those events flow through the StreamWriter into the orchestration layer.

```
Your tool code                    Framework internals
     |                                 |
     v                                 v
 StreamEmitter.block()  -->  StreamWriter  -->  orchestration.py
 StreamEmitter.thinking()                       project_chunk()
                                                    |
                                                    v
                                             AG-UI SSE event
                                             to the browser
```

StreamReducer (`api/chat/stream_reducer.py`) -- Sits in the orchestration layer. It watches every AG-UI event that goes out and accumulates a snapshot: all the text, blocks, and workflow state from the current run. When the stream ends, this snapshot gets persisted as the assistant message.

You don't interact with either of these directly. They exist so that:

- Tools can stream progress mid-execution (via `get_emitter(config)`)
- Workflows can stream step updates (via the SDK's `StepProgress`)
- Everything gets persisted automatically when the stream ends
- The frontend can reload and see the full conversation history

2. Where Things Live

```
src/agentic_platform/
  app/        <-- YOUR CODE. Tools, workflows, blocks, prompts.
  core/       <-- Framework. Don't touch unless you're building engine features.
  api/        <-- HTTP layer. Routes, SSE streaming, AG-UI projection.
```

Frontend: `src/features/chatv2/components/blocks/` -- your block components go here.

---

3. Adding a Tool

You need three things: the function, a wrapper, and registration.

```python
# app/campaigns/tools/my_tool.py
from langchain_core.tools import tool
from src.agentic_platform.core.engine import AgenticTool, ToolResponse, ToolTag

@tool("get_campaign_roi")
async def _get_campaign_roi(campaign_id: str) -> dict:
    """Get ROI metrics for a campaign. Call this when the user asks
    about campaign performance, returns, or efficiency."""
    data = await fetch_roi(campaign_id)
    return ToolResponse(
        summary=f"Campaign {campaign_id}: {data['roi']}% ROI, ${data['spend']}
spent",
        data=data,
    ).model_dump()

get_campaign_roi = AgenticTool(
    func=_get_campaign_roi,
    thinking_messages=["Pulling ROI data..."],
    tags=[ToolTag.ANALYTICS],
)
```

Register it:

```python
# app/campaigns/agent.py
config = AgentConfig(
    ...
    tools=[get_campaign_roi, ...],
)
```

The docstring is everything. The LLM reads it to decide when to call your tool. Write it like you're explaining to a smart coworker when this tool is useful.

Adding Rich UI to a Tool

If you want your tool to render a card/table/widget instead of just text:

```python
# 1. Define the block schema (app/campaigns/blocks.py)
from src.agentic_platform.core.engine import BlockSpec, register_block_spec,
BlockDisplay

class ROIData(BaseModel):
    campaign_id: str
    roi_percent: float
    spend: float

roi_block = register_block_spec(BlockSpec(
    block_type="campaign_roi",        # Must match frontend BLOCK_REGISTRY key
    data_schema=ROIData,
    display=BlockDisplay.INLINE,      # INLINE | SIDEBAR | MODAL | FULLSCREEN
))

# 2. Emit it from your tool
return ToolResponse(
    summary=f"ROI is {data['roi']}%",
    ui_blocks=[roi_block.create(data=ROIData(**data))],
).model_dump()
```

```cpp
// 3. Frontend: features/chatv2/components/blocks/CampaignROI.tsx
export function CampaignROI({ data }) {
  return <div className="rounded-lg border p-4">ROI: {data.roi_percent}%</div>
}

// 4. Register: features/chatv2/components/blocks/BlockRegistry.tsx
campaign_roi: { inline: CampaignROI },
```

That's it. Backend emits the block, frontend renders it. The `block_type` string is the only coupling.

For sidebar/modal/fullscreen, you create two components: a trigger (the card in chat) and the content (what opens).

4. Adding a Workflow

Workflows are for multi-step operations like "create a campaign" -- research, analyze, plan, build, save.

```python
# app/campaigns/workflows/create_campaign.py
from langchain_core.tools import tool
from src.agentic_platform.core.engine import (
    AgenticWorkflow, Step, Parallel, SubStep, WorkflowContext, NodeResponse,
)

# 1. Trigger tool -- LLM calls this to start the workflow
@tool("create_media_plan")
async def _create_media_plan(url: str, budget: float) -> dict:
    """Create a full media plan for an advertiser. Use when the user
    wants to create or build a new campaign."""
    raise NotImplementedError  # Framework routes to the workflow graph

# 2. Step functions -- each receives WorkflowContext
async def research(ctx: WorkflowContext) -> NodeResponse:
    data = await scrape(ctx.args["url"])
    return NodeResponse(summary=f"Scraped {len(data)} pages", data=data)

async def analyze(ctx: WorkflowContext) -> NodeResponse:
    scraped = ctx.results["research"].data       # prior step output
    ctx.progress.start("market")                  # substep progress
    market = await analyze_market(scraped)
    ctx.progress.done("market", summary="3 competitors found")
    ctx.emit_artifact(StepArtifact(               # insights sidebar
        type="market_context", title="Market", data=market
    ))
    return NodeResponse(summary="Analysis complete", data=market)

# 3. Assemble the workflow
create_media_plan = AgenticWorkflow(
    trigger=_create_media_plan,
    title="Creating media plan for {url}",
    steps=[
        Step("research", research, label="Research website"),
        Step("analyze", analyze, label="Analyze market", substeps=[
            SubStep("market", "Market analysis"),
            SubStep("brand", "Brand analysis"),
        ]),
        Step("plan", plan_step, label="Generate plan"),
        Step("save", save_step, label="Save"),
    ],
)
```

Key things in WorkflowContext:

- `ctx.args` -- trigger arguments (`{"url": "...", "budget": 50000}`)
- `ctx.results` -- prior step outputs (`{"research": NodeResponse(...)}`)

- `ctx.progress` -- substep progress (`.start("name")` , `.done("name", summary)`)
- `ctx.emit_artifact(...)` -- structured insights for the sidebar
- `ctx.org_id` , `ctx.user_id` -- current user context

HITL in Workflows

To pause and ask the user for input:

```
from src.agentic_platform.core.engine import HITLRequest, HITLActionButton,
HITLField, HITLAction

async def configure(ctx: WorkflowContext) -> NodeResponse:
    plan = ctx.results["plan"].data
    return NodeResponse(
        summary="Ready for review",
        data=plan,
        hitl=HITLRequest(
            type="form",
            title="Review campaign settings",
            fields=[
                HITLField(name="budget", label="Budget", type="number",
default=plan["budget"]),
                HITLField(name="start_date", label="Start Date", type="date"),
            ],
            actions=[
                HITLActionButton(action=HITLAction.SUBMIT, label="Approve",
style="primary"),
                HITLActionButton(action=HITLAction.REJECT, label="Cancel"),
            ],
        ),
    )
```

The workflow pauses, the frontend renders the form, the user submits, and the next step reads `ctx.results["configure"].metadata["user_input"]` .

5. Adding a New Agent

```python
# 1. Create app/my_domain/agent.py
config = AgentConfig(
    agent_id="my-agent",
    name="My Agent",
    system_prompt="You are...",
    tools=[my_tool],
    workflows=[my_workflow],
)

# 2. Register in app/__init__.py
def get_all_agent_configs():
    from src.agentic_platform.app.campaigns.agent import config as campaigns
    from src.agentic_platform.app.my_domain.agent import config as my_agent
    return [campaigns, my_agent]
```

The agent is now available at `/api/v1/agents/my-agent/chat/`.

---

6. Frontend Quick Reference

How Streaming Works

```
User sends message
  -> POST /api/chatv2/agents/{id}/chat/{conv}
  -> BFF adds JWT + org headers, pipes SSE to browser
  -> sseParser splits raw text into AG-UI events
  -> Redux reducer (chatSlice.ts) processes each event:

    RUN_STARTED          ->  streaming = true
    TEXT_MESSAGE_CONTENT  ->  append to streamingText
    STEP_STARTED/FINISHED ->  update activeSteps
    ACTIVITY_SNAPSHOT     ->  set workflow (progress card)
    ACTIVITY_DELTA        ->  patch workflow (step status change)
    CUSTOM (block)        ->  append to streamingBlocks
    CUSTOM (hitl_request) ->  set pendingHITL, pause
    RUN_FINISHED          ->  commit message, streaming = false
```

Adding a Frontend Block

```tsx
// 1. Create component
export function MyBlock({ data }: { data: MyData }) {
  return <div>{data.title}</div>
}

// 2. Register in BlockRegistry.tsx
my_block_type: { inline: MyBlock },
```

For sidebar blocks, register both a trigger and content:

```
my_details: {
  inlineTrigger: MyDetailsTrigger,   // clickable card in chat
  sidebar: MyDetailsContent,          // opens in right panel
},
```

Adding an Artifact Renderer

```
// components/workflow/artifact-renderers/MyRenderer.tsx
export function MyRenderer({ data }) { return <div>...</div> }

// Register in artifact-renderers/index.ts
my_artifact_type: MyRenderer,
```

7. Rules of the Road

Rule	Why
Tool docstring = tool's contract with the LLM	Bad docstring = LLM calls it at wrong times
Always return `ToolResponse(...).model_dump()`	Framework handles all streaming
`summary` for LLM, `ui_blocks` for humans	LLM never sees blocks, humans see both
Never use `os.getenv()`, use `settings.xxx`	Single source of truth in `core/config.py`
Never create HTTP clients, use `ServiceClient`	Circuit breakers, retries, metrics
`block_type` string must match frontend registry	Only coupling between backend and frontend
Tool name in `@tool("name")` -- no underscores	Function name `_name` with underscore (convention)

---

8. Troubleshooting

Symptom	Check
LLM never calls my tool	Is the docstring clear? Is it registered in AgentConfig?
Block not rendering	Does `block_type` match `BLOCK_REGISTRY` key exactly?
SSE stream drops	Check BFF proxy. Is `REACT_AGENTIC_SERVICE_URL` set?
HITL not pausing	Is `NodeResponse.hitl` set? Is it an `HITLRequest` instance?
Workflow steps not updating	Check `ACTIVITY_DELTA` events in browser Network tab
Tool timeout	Default is 30s for tools, 60s for steps. Increase if needed.
Config not picked up	Added to `core/config.py` Settings class? In `.env`?

Quick Debug

```
curl http://localhost:8080/health
curl http://localhost:8080/api/v1/agents/ | jq
curl http://localhost:8080/api/v1/block-schemas | jq
curl -N http://localhost:8080/api/v1/agents/campaign-assistant/chat/ \
  -H "Content-Type: application/json" \
  -d '{"messages":
[{"id":"1","role":"user","content":"hello"}],"threadId":"t","runId":"r"}'
```